КУРСОВАЯ РАБОТА

по дисциплине «Алгоритмы и структуры данных»

Тема: «Реализация игры “2048” с динамическим увеличением игрового поля»

Автор: Кутепов П.О.

СПбГЭТУ «ЛЭТИ», кафедра САПР

2025

Оглавление

Оглавление 2

Введение 3

Структуры данных 3

Игровое поле (grid) 3

Ячейка (cell) 4

Плитка (tile) 5

Основные алгоритмы 5

Инициализация игрового поля 5

Пользовательский ввод 6

Добавление новой плитки 7

Сдвиг и слияние плиток 7

Расширение игрового поля 9

Проверка возможности сделать ход 11

Алгоритмическая сложность игрового хода 11

Текст программы 12

index.html 12

script.js 12

grid.js 15

cell.js 17

tile.js 18

style.css 18

Заключение 19

Введение

В данной курсовой работе будет рассмотрена реализация модифицированной версии игры «2048». Ключевая цель игры остаётся неизменной — получение плитки номиналом 2048 путём сдвига всех плиток в одном из четырёх направлений и их последующего слияния при совпадении значений. Каждый ход сопровождается появлением в случайной свободной ячейке плитки со значением 2 или 4.

Основная модификация заключается в динамическом изменении размера игрового поля. Игра начинается на поле размером 2x2. При достижении значений 16, 64, 256 и 1024 размер поля увеличивается до 3x3, 4x4, 5x5 и 6x6 соответственно.

Для реализации игры будут разработаны следующие структуры данных: сетка (игровое поле), ячейка (элемент сетки) и плитка. Ключевые алгоритмы будут включать в себя: инициализацию и масштабирование игрового поля, механизм сдвига, слияния плиток и генерации новой плитки в случайной позиции.

Структуры данных

Игровое поле (grid)

Поле реализовано на основе системы массивов (рис. 1). Базовой структурой является одномерный массив cells, хранящий ссылки на все ячейки поля. Для эффективного выполнения операций со сдвигом массив cells проецируется на четыре двумерных представления: cellsGroupedByColumn — массив столбцов; cellsGroupedByReversedColumn — массив столбцов в обратном порядке (снизу вверх); cellsGroupedByLine — массив строк; cellsGroupedByReversedLine — массив строк в обратном порядке (справа налево). Размерность игрового поля контролируется переменной gridSize.

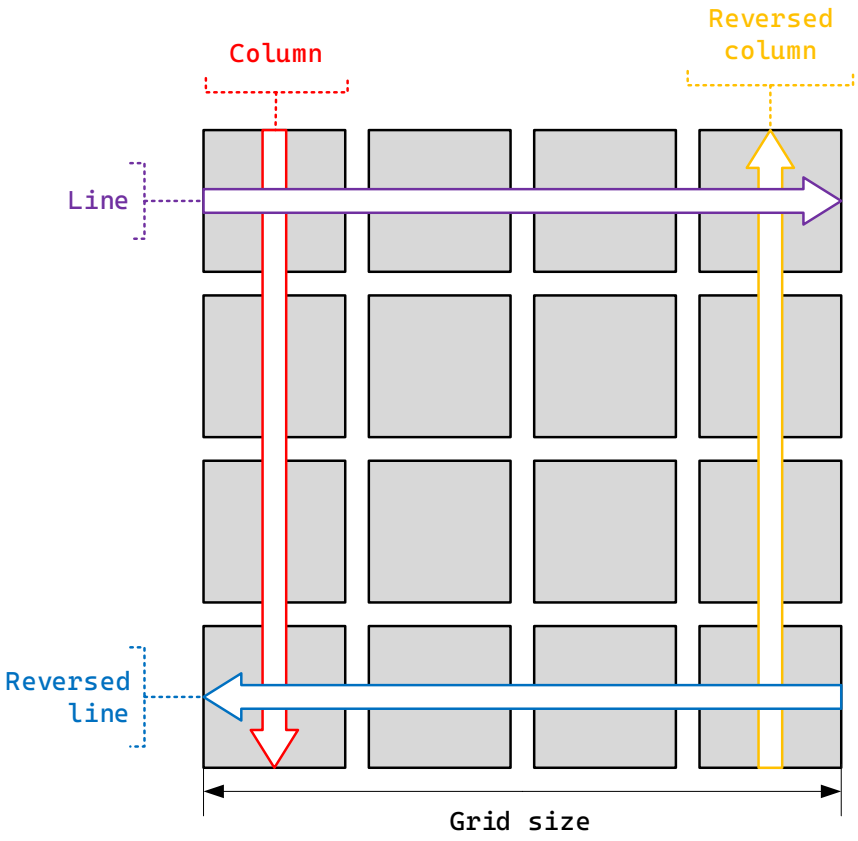


Рисунок 1 – Структура игрового поля

Ячейка (cell)

Ячейка содержит свою графическую реализацию в переменной cell, хранит координаты x, y (номер столбца, номер строки) и прикреплённую за ней плитку linkedTile (рис. 2).

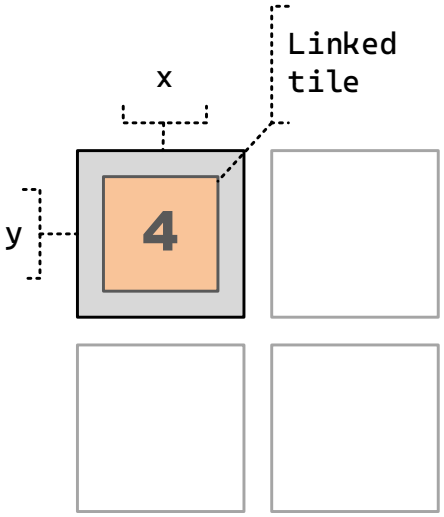


Рисунок 2 – Ячейка

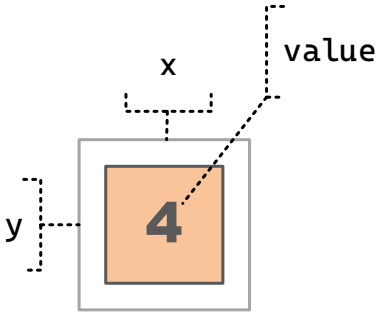


Рисунок 3 – Плитка

Плитка (tile)

Плитка содержит данные о своём внешнем виде в переменной tile, расположении (координаты x, y, за которую она прикреплена) и значении value (рис. 3).

Основные алгоритмы

Инициализация игрового поля

Алгоритм инициализации сетки начинается с присвоения переменной gameBoard области на экране с id «game-board». Затем создаётся объект класса Grid, принимающий эту область и размерность сетки.

На этапе инициализации Grid два вложенных цикла (по осям Y и X) создают все ячейки игрового поля. Каждая ячейка привязывается к gameBoard, получает свои координаты, графический стиль и помещается в основной одномерный массив cells.

После этого функциями (groupCellByColumn, groupCellByLine и др.) формируются четыре логических представления сетки для оптимизации алгоритмов сдвига: column (столбцы), reversedColumn (столбцы снизу вверх), line (строки) и reversedLine (строки справа налево). Заполнение этих массивов также происходит в циклах, где порядок обхода координат адаптируется под нужное направление (например, для обратных массивов обход начинается с конца).

$T(n)$	$S(n)$	Обоснование
Grid constructor		
$\Theta(n^2)$	$\Theta(1)$	В любом случае создастся n^2 ячеек
Cell constructor		
$\Theta(1)$	0	Процесс создания ячейки не зависит от входных данных
Grid.groupCellsByColumn, Grid.groupCellsByReversedColumn, Grid.groupCellsByLine, Grid.groupCellsByReversedLine		
$\Theta(n^2)$	$\Theta(n)$	Полностью перебирается массив ячеек для их распределения
Общая сложность алгоритма		
$\Theta(n^2)$	$\Theta(n)$	$2\Theta(n^2) + \Theta(1) = \Theta(n^2)$ $\Theta(1) + \Theta(n) = \Theta(n)$

Таблица 1 – Оценка сложности функций и всего алгоритма (n = gridSize)

Пользовательский ввод

Функция isMobileDevice определяет, использует ли пользователь устройство с тачскрином.

Если устройство не сенсорное, вызывается функция setupInputByKeyboard. Она однократно подписывается на событие нажатия клавиши. При срабатывании события функция определяет направление по нажатой стрелке, передаёт его в функцию makeMove для обработки хода и рекурсивно вызывает саму себя, чтобы ожидать следующего действия пользователя.

Для мобильных устройств функция setupInputByTouchscreen отслеживает начало и конец касания. При начале касания фиксируются координаты startX и startY. Когда касание завершается, по разнице координат (diffX, diffY) вычисляется направление свайпа (рис. 4), которое передаётся в makeMove, и функция переустанавливается для следующего жеста.

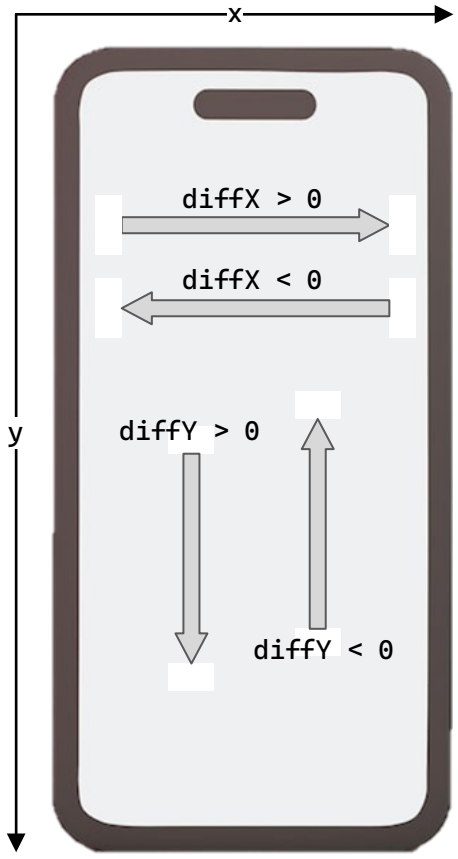


Рисунок 4 – Определение направления свайпа по разнице координат

$T(n)$	$S(n)$	Обоснование
isMobileDevice		
$\Theta(1)$	$\Theta(1)$	Проверка одного условия
setupInputByKeyboard		
$\Theta(1)$	0	Единожды устанавливается обработчик события нажатия клавиши
setupInputByTouchscreen		
$\Theta(1)$	$\Theta(1)$	Единожды устанавливаются события начала и конца касания
Общая сложность алгоритма		
$\Theta(1)$	$\Theta(1)$	-

Таблица 2

Добавление новой плитки

За добавление новой плитки со случайным значением в случайном месте отвечает метод getRandomEmptyCell класса Grid. В emptyCells заносятся все пустые ячейки. Math.random генерирует случайное число в диапазоне $[0; 1)$. Случайная ячейка выбирается домножением на Math.random длины массива emptyCell, значение для неё выбирается условием: если Math.random возвращает >0.5 value становится равным 2, иначе 4.

$T(n)$	$S(n)$	Обоснование
Tile constructor		
$\Theta(1)$	0	Процесс создания плитки не зависит от входных данных
Array.filter		
$\Theta(n^2)$	0	Проверка всех n^2 ячеек
Grid.getRandomEmptyCell		
$\Theta(n^2)$	$O(n^2)$	Проверка всех n^2 ячеек и создание массива под пустые
Общая сложность алгоритма		
$\Theta(n^2)$	$O(n^2)$	-

Таблица 3 - Оценка сложности функций и всего алгоритма (n = gridSize)

Сдвиг и слияние плиток

Алгоритм сдвига состоит из нескольких шагов. Функция makeMove определяет направление и передает в slideTiles соответствующий массив строк или столбцов. Также makeMove отслеживает максимальную плитку,

полученную пользователем, при помощи метода findMaxValue, и, если на поле имеется плитка 2048, игра выведет сообщение о победе.

Функция slideTiles перебирает все ячейки переданного массива по двум циклам. Для каждой непустой ячейки в линии определяется целевая ячейка (рис. 5) — самая дальняя ячейка в направлении движения, которая может принять плитку. Функция canAccept проверяет, является ли целевая ячейка пустой или содержит плитку с таким же значением.

Если целевая ячейка найдена, происходит либо перемещение плитки (метод linkTile), либо слияние (метод mergeTiles). Ячейка targetCell заносится в массив usedTiles, чтобы исключить множественные слияния за один ход. После перемещения исходная ячейка очищается.

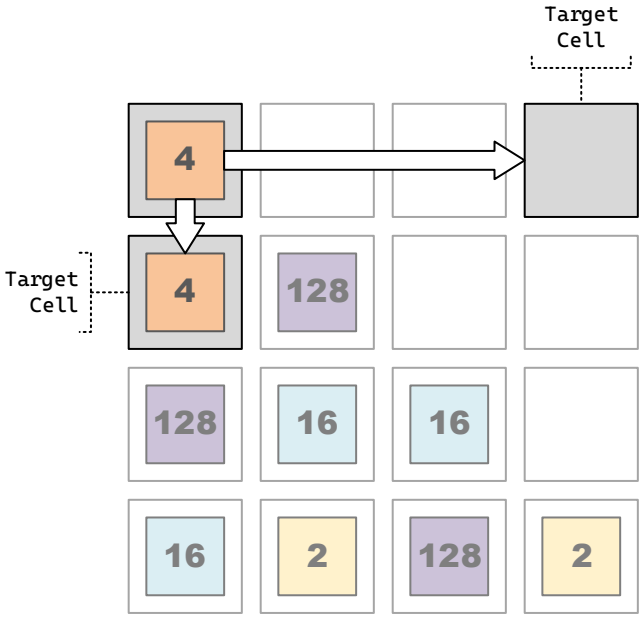


Рисунок 5 – Поиск целевой ячейки

Case	$T(n)$	$S(n)$	Обоснование
findMaxValue			
Best	$\Theta(n^2)$	$\Theta(1)$	Проверка всех n^2 ячеек и хранение одного максимального значения
Average	$\Theta(n^2)$	$\Theta(1)$	
Worst	$\Theta(n^2)$	$\Theta(1)$	
Array.includes			
Best	$\Theta(1)$	0	Искомый элемент в начале
Average	$O(n^2)$	0	Искомый элемент в середине массива

Worst	$\Theta(n^2)$	0	Совпадений нет
slideTiles			
Best	$\Theta(n^2)$	$O(n)$	Все ячейки пустые
Average	$\Omega(n^2)$~$O(n^3)$	$O(n)$	30-70% заполнения поля
Worst	$\Theta(n^3)$	$O(n)$	Максимальное перемещение
Общая сложность алгоритма			
Best	$\Theta(n^2)$	$O(n)$	Выбирается наибольшая оценка
Average	$\Omega(n^2)$~$O(n^3)$	$O(n)$	
Worst	$\Theta(n^3)$	$O(n)$	

Таблица 4 – Оценка сложности функций и всего алгоритма (n = gridSize)

Расширение игрового поля

Увеличение размера сетки выполняется методом increaseGrid. При этом текущее состояние сетки сохраняется в oldGrid, а затем создается новый объект grid с увеличенной на 1 размерностью. Пороговое значение valueForExtension для следующего увеличения умножается на 4.

После этого специальные методы (moveUpTiles, moveDownTiles и др.) переносят все плитки из oldGrid в новую сетку. Каждый из этих методов определяет правило смещения координат (например, сдвиг по оси X на 1), чтобы корректно расположить плитки в расширенном поле.

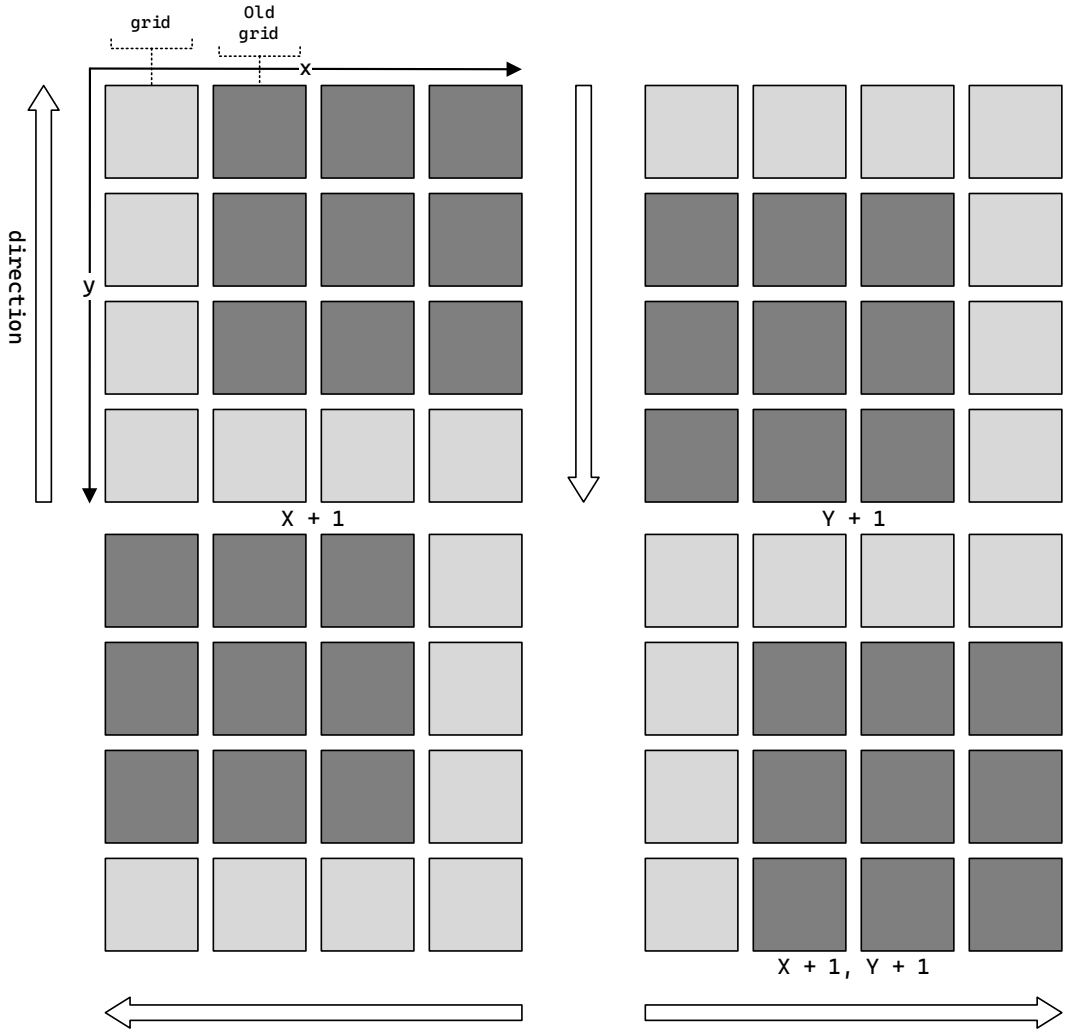


Рисунок 6 – Сдвиг плиток в зависимости от направления

$T(n)$	$S(n)$	Обоснование
Grid.moveUpTiles, Grid.moveDownTiles, Grid.moveLeftTiles, Grid.moveRightTiles		
$\Theta(n^2)$	$\Theta(1)$	Перебор всех ячеек для перемещения плиток
increasingGrid		
$\Theta(n^2)$	$\Theta(1)$	Вызов функций, перемещающих плитки
Создание новой сетки		
$\Theta(n^2)$	$\Theta(n)$	Алгоритм инициализации игрового поля
Общая сложность алгоритма		
$\Theta(n^2)$	$\Theta(n)$	-

Таблица 5 – Оценка сложности функций и всего алгоритма (n = gridSize)

Проверка возможности сделать ход

Функция canMove проверяет, возможно ли сделать ход в пределах одного направления. Для этого она перебирает все ячейки. Если находится хотя бы одна непустая ячейка, рядом с которой есть пустая ячейка или ячейка с таким же значением плитки, функция немедленно возвращает true. Если таких пар не обнаружено во всех направлениях, функция возвращает false, и игра предлагает пользователю начать заново.

Case	$T(n)$	$S(n)$	Обоснование
canMove			
Best	$\Theta(1)$	$\Theta(n)$	Ход находится сразу
Average	$O(n^2)$	$\Theta(n)$	30-70% заполнения поля
Worst	$\Theta(n^2)$	$\Theta(n)$	Ход сделать нельзя
Общая сложность алгоритма			
Best	$\Theta(1)$	$\Theta(n)$	Хранение групп ячеек и поиск тех ячеек, которые можно сдвинуть
Average	$O(n^2)$	$\Theta(n)$	
Worst	$\Theta(n^2)$	$\Theta(n)$	

Таблица 6 - Оценка сложности функций и всего алгоритма (n = gridSize)

Алгоритмическая сложность игрового хода

Обработка хода оценивается исходя из размера игрового поля gridSize, и состоит из последовательных этапов. Изначально проверяется возможность хода. Если ход невозможен, выполняется полный перебор всех ячеек, что в лучшем случае дает $\Theta(n^2)$. Далее игра ожидает следующий ввод. Если ход возможен, производится сдвиг и слияние плиток. Сложность данной операции от $\Theta(n^2)$ в лучшем случае (все ячейки пусты) до $\Theta(n^3)$ в худшем (требуется максимальное перемещение плиток) (табл. 6). Далее производится поиск максимальной плитки для проверки победы. По необходимости требуется увеличить размер поля. Генерация новой плитки в случайной пустой ячейке имеет сложность $\Theta(n^2)$ по времени и $O(n^2)$ по памяти, так как требует создания массива пустых ячеек. В конце проверяется возможность следующего хода в четырех направлениях. Её сложность составляет от $\Theta(1)$ до $\Theta(n^2)$, в

зависимости от того, будет ли возможный ход найден сразу или потребуется полный обход поля.

Итоговая сложность одного хода равняется сумме сложностей этих последовательных этапов и определяется наиболее затратной операцией – сдвигом и добавлением новой плитки, – что дает оценку от $\Theta(n^2)$ до $\Theta(n^3)$.

Этап	$T(n)$			$S(n)$
	Best	Average	Worst	
Пользовательский ввод	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$
Проверка возможности хода	$\Theta(1)$	$O(n^2)$	$\Theta(n^2)$	$\Theta(n)$
Сдвиг и слияние плиток	$\Theta(n^2)$	$\Omega(n^2)\sim O(n^3)$	$\Theta(n^3)$	$O(n)$
Расширение игрового поля	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n)$
Добавление новой плитки	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(n^2)$
Проверка возможности хода× 4	$\Theta(1)$	$O(n^2)$	$\Theta(n^2)$	$\Theta(n)$
ИТОГО	$\Theta(n^2)$	$\Omega(n^2)\sim O(n^3)$	$\Theta(n^3)$	$O(n^2)$

Таблица 6 – Оценка хода (n = gridSize)

Текст программы

index.html

```
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>2048</title>
    <link rel="stylesheet" href="./style.css">
    <script type="module" src="./script.js"></script>
</head>

<body>
    <div id="game-board"></div>
</body>

        </html>
```

script.js

```
import { Grid } from "./grid.js";
import { Tile } from "./tile.js"

const gameBoard = document.getElementById("game-board")

let gridSize = 2
let valueForExtension = 16
let grid = new Grid(gameBoard, gridSize)

grid.getRandomEmptyCell().linkTile(new Tile(gameBoard))
if (isMobileDevice()) {
    document.addEventListener('touchmove', function (event) {
                if (window.scrollY === 0) {
                    event.preventDefault();
                }
            }, { passive: false });
```

```
    setupInputByTouchscreen()
} else {
    setupInputByKeyboard()
}

function setupInputByKeyboard() {
    window.addEventListener("keydown", (event) => {
        makeMove(event.key.slice(5))
        setupInputByKeyboard()
    }, { once: true })
}

function setupInputByTouchscreen() {
    let startX
    let startY
    window.addEventListener("touchstart", (event) => {
        startX = event.touches[0].clientX;
        startY = event.touches[0].clientY;
    }, { once: true })
    let diffX
    let diffY
    window.addEventListener("touchend", (event) => {
        diffX = event.changedTouches[0].clientX - startX;
        diffY = event.changedTouches[0].clientY - startY;
        if (Math.abs(diffY) > Math.abs(diffX)) {
            if (diffY < 0) {
                makeMove("Up")
                setupInputByTouchscreen()
            }
            else {
                makeMove("Down")
                setupInputByTouchscreen()
            }
        } else {
            if (diffX < 0) {
                makeMove("Left")
                setupInputByTouchscreen()
            }
            else {
                makeMove("Right")
                setupInputByTouchscreen()
            }
        }
    }, { once: true })
}

async function makeMove(direction) {
    switch (direction) {
        case "Up":
            if (!canMoveUp()) {
                return
            }
            moveUp();
            break
        case "Down":
            if (!canMoveDown()) {
                return
            }
            moveDown();
            break
        case "Left":
            if (!canMoveLeft()) {
                return
            }
            moveLeft()
            break
        case "Right":
            if (!canMoveRight()) {
                return
            }
            moveRight()
            break
    }

    await new Promise(resolve => setTimeout(resolve, 100))
```

```
    const maxValue = grid.findMaxValue()
    if (maxValue === 2048) {
        await new Promise(resolve => setTimeout(resolve, 100))
        alert("YOU WON!")
        return
    } else if (maxValue === valueForExtension) {
        increasingGrid(direction)
    }

    grid.getRandomEmptyCell().linkTile(new Tile(gameBoard))

    if (!canMoveUp() && !canMoveDown() && !canMoveLeft() && !canMoveRight()) {
        await new Promise(resolve => setTimeout(resolve, 100))
        alert("Try again!")
        location.reload(true)
        return
    }
}

function moveUp() {
    slideTiles(grid.cellsGroupedByColumn)
}

function moveDown() {
    slideTiles(grid.cellsGroupedByReversedColumn)
}

function moveLeft() {
    slideTiles(grid.cellsGroupedByLine)
}

function moveRight() {
    slideTiles(grid.cellsGroupedByReversedLine)
}

function slideTiles(groupedCells) {
    for (let i = 0; i < groupedCells.length; i++) {
        const group = groupedCells[i]

        let usedCells = []

        for (let j = 1; j < group.length; j++) {
            if (group[j].isEmpty()) {
                continue
            }
            const cellWithTile = group[j]

            let targetCell
            for (let k = j - 1; k >= 0 && group[k].canAccept(cellWithTile.linkedTile); k--) {
                targetCell = group[k]
            }

            if (!targetCell) {
                continue
            } else if (targetCell.isEmpty()) {
                targetCell.linkTile(cellWithTile.linkedTile)
            } else if (usedCells.includes(targetCell)) {
                continue
            } else {
                targetCell.mergeTiles(cellWithTile.linkedTile)
                usedCells.push(targetCell)
            }
            cellWithTile.linkedTile = null
        }
    }
}

function canMoveUp() {
    return canMove(grid.cellsGroupedByColumn)
}

function canMoveDown() {
    return canMove(grid.cellsGroupedByReversedColumn)
}
```

```
function canMoveLeft() {
    return canMove(grid.cellsGroupedByLine)
}

function canMoveRight() {
    return canMove(grid.cellsGroupedByReversedLine)
}

function canMove(groupedCells) {
    for (let i = 0; i < groupedCells.length; i++) {
        const group = groupedCells[i]

        for (let j = 1; j < group.length; j++) {
            if (group[j].isEmpty()) {
                continue
            }
            const cellWithTile = group[j]

            const targetCell = group[j - 1]
            if (targetCell.canAccept(cellWithTile.linkedTile)) {
                return true
            }
        }
    }
    return false
}

function increasingGrid(direction) {
    gridSize++
        valueForExtension *= 4

        const oldGrid = grid

        grid = new Grid(gameBoard, gridSize)
        switch (direction) {
            case "Up":
                grid.moveUpTiles(oldGrid)
                break
            case "Down":
                grid.moveDownTiles(oldGrid)
                break
            case "Left":
                grid.moveLeftTiles(oldGrid)
                break
            case "Right":
                grid.moveRightTiles(oldGrid)
                break
        }

        oldGrid.removeFromDOM()
}

function isMobileDevice() {
    const userAgent = navigator.userAgent.toLowerCase();

    const mobileKeywords = [
        'android', 'iphone', 'ipod', 'windows phone', 'mobile', 'tablet', 'ipad'
    ];

    return mobileKeywords.some(keyword => userAgent.includes(keyword));
        }
```

grid.js

```
import { Cell } from "./cell.js";

export class Grid {
    gridSize
    cells = []
    cellsGroupedByColumn = []
    cellsGroupedByLine = []
    cellsGroupedByReversedColumn = []
    cellsGroupedByReversedLine = []

    constructor(gridElement, gridSize) {
```

```
        this.gridSize = gridSize
        gridElement.style.setProperty("--grid-size", gridSize)

        for (let i = 0; i < this.gridSize; i++) {
            for (let j = 0; j < this.gridSize; j++) {
                this.cells.push(new Cell(gridElement, j, i))
            }
        }

        this.groupCellByColumn()
        this.groupCellByLine()
        this.groupCellByReversedColumn()
        this.groupCellByReversedLine()
    }

    getRandomEmptyCell() {
        const emptyCells = this.cells.filter(cell => cell.isEmpty())
        const randomIndex = Math.floor(Math.random() * emptyCells.length)
        return emptyCells[randomIndex]
    }

    groupCellByColumn() {
        for (let i = 0; i < this.gridSize; i++) {
            const column = []
            for (let j = 0; j < this.gridSize; j++) {
                column.push(this.cells[this.gridSize * j + i])
            }
            this.cellsGroupedByColumn.push(column)
        }
    }
    groupCellByLine() {
        for (let i = 0; i < this.gridSize; i++) {
            const line = []
            for (let j = 0; j < this.gridSize; j++) {
                line.push(this.cells[this.gridSize * i + j])
            }
            this.cellsGroupedByLine.push(line)
        }
    }
    groupCellByReversedColumn() {
        for (let i = 0; i < this.gridSize; i++) {
            const reversedColumn = []
            for (let j = this.gridSize - 1; j >= 0; j--) {
                reversedColumn.push(this.cells[this.gridSize * j + i])
            }
            this.cellsGroupedByReversedColumn.push(reversedColumn)
        }
    }
    groupCellByReversedLine() {
        for (let i = 0; i < this.gridSize; i++) {
            const reversedLine = []
            for (let j = this.gridSize - 1; j >= 0; j--) {
                reversedLine.push(this.cells[this.gridSize * i + j])
            }
            this.cellsGroupedByReversedLine.push(reversedLine)
        }
    }

    findMaxValue() {
        let maxValue = 0
        this.cells.forEach(cell => {
            if (cell.linkedTile) {
                maxValue = maxValue < cell.linkedTile.value ? cell.linkedTile.value : maxValue
            }
        })
        return maxValue
    }

    moveUpTiles(oldGrid) {
        for (let i = 0; i < oldGrid.gridSize; i++) {
            for (let j = 0; j < oldGrid.gridSize; j++) {
                if (oldGrid.cells[i * oldGrid.gridSize + j].linkedTile) {
                    this.cells[i * this.gridSize + (j + 1)].linkTile(oldGrid.cells[i *
oldGrid.gridSize + j].linkedTile)
                }
```

```
            }
        }
    }

    moveDownTiles(oldGrid) {
        for (let i = 0; i < oldGrid.gridSize; i++) {
            for (let j = 0; j < oldGrid.gridSize; j++) {
                if (oldGrid.cells[i * oldGrid.gridSize + j].linkedTile) {
                    this.cells[(i + 1) * this.gridSize + j].linkTile(oldGrid.cells[i *
oldGrid.gridSize + j].linkedTile)
                }
            }
        }
    }

    moveLeftTiles(oldGrid) {
        for (let i = 0; i < oldGrid.gridSize; i++) {
            for (let j = 0; j < oldGrid.gridSize; j++) {
                if (oldGrid.cells[i * oldGrid.gridSize + j].linkedTile) {
                    this.cells[i * this.gridSize + j].linkTile(oldGrid.cells[i * oldGrid.gridSize +
j].linkedTile)
                }
            }
        }
    }

    moveRightTiles(oldGrid) {
        for (let i = 0; i < oldGrid.gridSize; i++) {
            for (let j = 0; j < oldGrid.gridSize; j++) {
                if (oldGrid.cells[i * oldGrid.gridSize + j].linkedTile) {
                    this.cells[(i + 1) * this.gridSize + (j + 1)].linkTile(oldGrid.cells[i *
oldGrid.gridSize + j].linkedTile)
                }
            }
        }
    }

    removeFromDOM() {
        this.cells.forEach(cell => {
            cell.cell.remove()
        })
    }
        }
```

cell.js

```
export class Cell {
    cell
    x
    y
    linkedTile

    constructor(gridElement, x, y) {
        this.cell = document.createElement("div")
        this.cell.classList.add("cell")
        this.x = x
        this.y = y
        gridElement.append(this.cell)
    }

    linkTile(tile) {
        tile.setXY(this.x, this.y)
        this.linkedTile = tile
    }

    isEmpty() {
        return !this.linkedTile
    }

    canAccept(newTile) {
        return (this.isEmpty() || this.linkedTile.value === newTile.value)
    }

    async mergeTiles(tile) {
        tile.setXY(this.x, this.y)
        await new Promise(resolve => setTimeout(resolve, 100))
```

```
        this.linkedTile.setValue(this.linkedTile.value + tile.value)
        tile.tile.remove()
    }
        }
```

tile.js

```
export class Tile {
    tile
    x
    y
    value

    constructor(gridElement) {
        this.tile = document.createElement("div")
        this.tile.classList.add("tile")
        this.setValue(Math.random() > 0.5 ? 2 : 4)
        gridElement.append(this.tile)
    }

    setXY(x, y) {
        this.x = x
        this.y = y
        this.tile.style.setProperty("--x", x)
        this.tile.style.setProperty("--y", y)
    }

    setValue(value) {
        this.value = value
        this.tile.textContent = value
        const color = Math.log2(value) * 34
        this.tile.style.setProperty("--bg-color", `hsl(${color}, 40%, 60%)`)
    }
        }
```

style.css

```
* {
    box-sizing: border-box;
    margin: 0;
    padding: 0;
}

body {
    background-color: #202020;
    display: flex;
    justify-content: center;
    align-items: center;
    height: 100vh;
}

@font-face {
    font-family: 'Archivo';
    src: url('ArchivoBlack-Regular.woff2') format('woff2');
    font-weight: 900;
    font-style: normal;
    font-display: swap;
}

#game-board {
    --grid-size: 2;
    --cell-size: calc(80vmin / var(--grid-size));
    --cell-gap: calc(10vmin / var(--grid-size));
    position: relative;
    display: grid;
    grid-template-columns: repeat(var(--grid-size), var(--cell-size));
    grid-template-rows: repeat(var(--grid-size), var(--cell-size));
    gap: var(--cell-gap);
    padding: var(--cell-gap);
    border-radius: 1vmin;
    background-color: #404040;
}

.cell {
    background-color: #202020;
    border-radius: 1vmin;
    animation: 100ms;
```

```
}

.tile {
    --x: 0;
    --y: 0;
    --bg-color: #ffffff;
    position: absolute;
    top: calc(var(--y) * (var(--cell-size) + var(--cell-gap)) + var(--cell-gap));
    left: calc(var(--x) * (var(--cell-size) + var(--cell-gap)) + var(--cell-gap));
    display: flex;
    justify-content: center;
    align-items: center;
    height: var(--cell-size);
    width: var(--cell-size);
    border-radius: 1vmin;
    font-family: 'Archivo';
    font-size: calc(0.35 * var(--cell-size));
    background-color: var(--bg-color);
    color: #202020;
    transition: 100ms;
    animation: show 100ms;
}

@keyframes show {
    0% {
        opacity: 0.5;
        transform: scale(0);
    }
        }
```

Заключение

В результате выполнения курсовой работы была успешно разработана и проанализирована модифицированная версия игры «2048» с динамическим увеличением игрового поля. В работе реализованы ключевые алгоритмы сдвига и слияния плиток, обеспечен кроссплатформенный ввод и проведен анализ вычислительной сложности. Было выявлено, что в данной реализации сложность одного хода составляет от $\Theta(n^2)$ до $\Theta(n^3)$, а пространственная сложность – $O(n^2)$. Работа демонстрирует практическую значимость знаний по алгоритмам и структурам данных для создания эффективных программных решений.